전북지역혁신플랫폼
Jeonbuk Regional Innovation Platform

JST

지능형모빌리티 설계

4주차 ROS2 프로그래밍 #2

3차시 | ROS2 Action



* 본 영상은 무단 배포할 수 없습니다.

전북대학교 전자공학부 | 조형기 교수

학습내용

1 Action 기반 ROS 구성

2 Action 사용 방법

학습목표

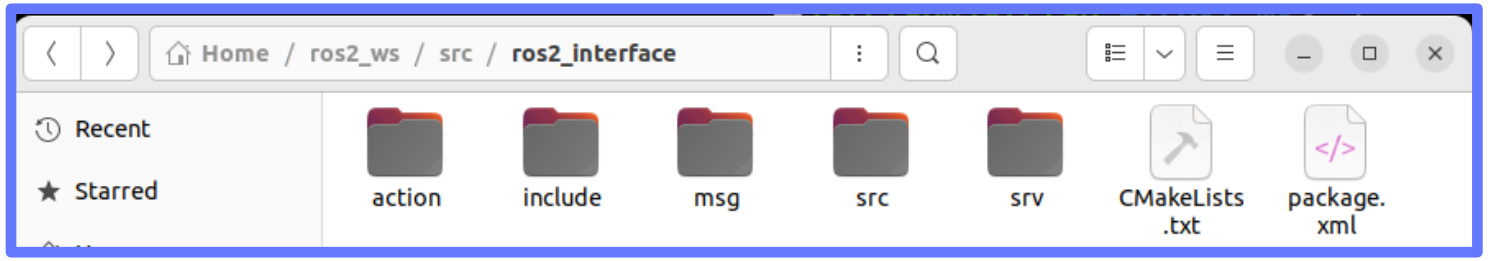
- Action 사용 실습을 통해 ROS 시스템을 만들어볼 수 있다.
- Action 결과를 확인할 수 있다.

Action 기반 ROS 구성

Action 생성

Action 디렉토리, 파일 생성

```
$ cd ~/ros2_ws/src/ros2_interface/
$ mkdir action
```



Action 생성

Action 디렉토리, 파일 생성

```
$ code Fibonnacci.action
```

Fibonacci.action

ros2_interface > action > Fibonacci.action

```
1   # Goal
2   int32 order
3
4   ---
5   # Result
6   int32[] sequence
7
8   ---
9   # Feedback
10  int32[] partial_sequence
11
```

Goal

- - -

Result

- - -

Feedback

Action 생성

피보나치 수열

피보나치 수: 첫째 및 둘째 항이 1이며,
그 뒤의 모든 항은 바로 앞 두 항의 합인 수열

- 0번째 항부터 시작할 경우 다음과 같이 정의됨

$F_0 = 0$
$F_1 = 1$
$F_n = F_{n-1} + F_{n-2}$ $(n \in \{2, 3, 4, \cdots\})$

예 0, 1, 1, 2, 3, 5, 8, 13, 21 ,34, 55, 89, ···

Action 생성

CMakeLists.txt

```
rosidl_generate_interfaces(${PROJECT_NAME}
"srv/ExCustomSrv.srv"
"action/Fibonnacci.action"
DEPENDENCIES
)
```

Action 파일

Action 생성

Build

```
$ cd ~/ros2_ws/
$ colcon build --packages-select ros2_interface
```

Action 생성

생성한 Action 확인

```
$ source install/setup.bash
```

```
$ ros2 interface show ros2_interface/action/Fibonnacci
```

출력

```
# Request
int32 order
---
# Result
int32[] sequence
---
# Feedback
int32[] partical_sequence
```

Action - cpp

action_server.hpp

```
$ cd ~/ros2_ws/src/ros2_cmake/include/ros2_cmake
$ code action_server.hpp
```

Action - cpp

action_server.hpp

```cpp
#ifndef ROS2_CMAKE__ACTION_SERVER_HPP_
#define ROS2_CMAKE__ACTION_SERVER_HPP_

#include <functional>
#include <memory>
#include <thread>
#include "ros2_interface/action/fibonacci.hpp"
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"

using Fibonacci = ros2_interface::action::Fibonacci;
using GoalHandleFibonacci = rclcpp_action::ServerGoalHandle<Fibonacci>;
```

Action - cpp

action_server.hpp

```
class FibonacciActionServer : public rclcpp::Node
{
public:
 explicit FibonacciActionServer(const rclcpp::NodeOptions & options = rcl
cpp::NodeOptions());
 virtual ~FibonacciActionServer() = default;
private:
 rclcpp_action::Server<Fibonacci>::SharedPtr action_server_;
 rclcpp_action::GoalResponse handle_goal(
   const rclcpp_action::GoalUUID & uuid,
   std::shared_ptr<const Fibonacci::Goal> goal);
 rclcpp_action::CancelResponse handle_cancel(
   const std::shared_ptr<GoalHandleFibonacci> goal_handle);
```

Action - cpp

action_server.hpp

```
  void handle_accepted(const std::shared_ptr<GoalHandleFibonacci> goal_handle);
  void execute(const std::shared_ptr<GoalHandleFibonacci> goal_handle);

};  // class FibonacciActionServer

#endif // ROS2_CMAKE__ACTION_SERVER_HPP_
```

Action - cpp

action_server.cpp

```
$ cd ~/ros2_ws/src/ros2_cmake/src/
$ code action_server.cpp
```

Action - cpp

action_server.cpp

```cpp
# include "ros2_cmake/action_server.hpp"

FibonacciActionServer::FibonacciActionServer(const rclcpp::NodeOptions & options )
 : Node("fibonacci_action_server", options)
 {
   using namespace std::placeholders;

   action_server_ = rclcpp_action::create_server<Fibonacci>(
     this,
     "fibonacci",
     std::bind(&FibonacciActionServer::handle_goal, this, _1, _2),
     std::bind(&FibonacciActionServer::handle_cancel, this, _1),
     std::bind(&FibonacciActionServer::handle_accepted, this, _1));
 }
```

Action - cpp

action_server.cpp

```
rclcpp_action::GoalResponse FibonacciActionServer::handle_goal(
   const rclcpp_action::GoalUUID & uuid,
   std::shared_ptr<const Fibonacci::Goal> goal)
 {
   RCLCPP_INFO(this->get_logger(), "Received goal request with order %d", goal->order);
   (void)uuid;
   return rclcpp_action::GoalResponse::ACCEPT_AND_EXECUTE;
 }
rclcpp_action::CancelResponse FibonacciActionServer::handle_cancel(
const std::shared_ptr<GoalHandleFibonacci> goal_handle)
{
RCLCPP_INFO(this->get_logger(), "Received request to cancel goal");
(void)goal_handle;
return rclcpp_action::CancelResponse::ACCEPT;
}
```

Action - cpp

action_server.cpp

```
void FibonacciActionServer::handle_accepted(const std::shared_ptr<GoalHandleFibonacci>
goal_handle)
{
using namespace std::placeholders;
// this needs to return quickly to avoid blocking the executor, so spin up a new thread
std::thread{std::bind(&FibonacciActionServer::execute, this, _1), goal_handle}.detach();
}
void FibonacciActionServer::execute(const std::shared_ptr<GoalHandleFibonacci> goal_handle)
{
   RCLCPP_INFO(this->get_logger(), "Executing goal");
   rclcpp::Rate loop_rate(1);
   const auto goal = goal_handle->get_goal();
   auto feedback = std::make_shared<Fibonacci::Feedback>();
   auto & sequence = feedback->partial_sequence;
```

Request 받기

Feedback 생성

Action - cpp

action_server.cpp

```
sequence.push_back(0);
  sequence.push_back(1);
  auto result = std::make_shared<Fibonacci::Result>();
  for (int i = 1; (i < goal->order) && rclcpp::ok(); ++i) {
      if (goal_handle->is_canceling()) {
      result->sequence = sequence;
      goal_handle->canceled(result);
      RCLCPP_INFO(this->get_logger(), "Goal canceled");
      return;
      }
      sequence.push_back(sequence[i] + sequence[i - 1]);
      goal_handle->publish_feedback(feedback);
      RCLCPP_INFO(this->get_logger(), "Publish feedback");
      loop_rate.sleep();
  }
```

피보나치 수열 두 번째 항까지 넣기

Result 생성

Goal 달성까지 수행

Feedback Publish

Action - cpp

action_server.cpp

```
// Check if goal is done
  if (rclcpp::ok()) {
      result->sequence = sequence;
      goal_handle->succeed(result);
      RCLCPP_INFO(this->get_logger(), "Goal succeeded");
  }
}
int main(int argc, char * argv[])
{
 rclcpp::init(argc, argv);
 auto action_server = std::make_shared<FibonacciActionServer>();
 rclcpp::spin(action_server);
 rclcpp::shutdown();
 return 0;
}
```

Result 보내기

Action - cpp

action_client.hpp

```
$ cd ~/ros2_ws/src/ros2_cmake/src/
$ code action_client.hpp
```

Action - cpp

action_client.hpp

```
#ifndef ROS2_CMAKE__ACTION_CLIENT_HPP_
#define ROS2_CMAKE__ACTION_CLIENT_HPP_

#include <functional>
#include <future>
#include <memory>
#include <string>
#include <sstream>
#include "ros2_interface/action/fibonacci.hpp"
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"

using Fibonacci = ros2_interface::action::Fibonacci;
using GoalHandleFibonacci = rclcpp_action::ClientGoalHandle<Fibonacci>;
```

Action - cpp

action_client.hpp

```
class FibonacciActionClient : public rclcpp::Node
{
public:

   explicit FibonacciActionClient(const rclcpp::NodeOptions & option
s = rclcpp::NodeOptions());
   virtual ~FibonacciActionClient() =default;
 void send_goal();
private:
 rclcpp_action::Client<Fibonacci>::SharedPtr client_ptr_;
 rclcpp::TimerBase::SharedPtr timer_;
```

Action - cpp

action_client.hpp

```
  void goal_response_callback(const GoalHandleFibonacci::SharedPtr & goal_handle);

  void feedback_callback(
    GoalHandleFibonacci::SharedPtr,
    const std::shared_ptr<const Fibonacci::Feedback> feedback);

  void result_callback(const GoalHandleFibonacci::WrappedResult & result);

};  // class FibonacciActionClient

#endif // ROS2_CMAKE__ACTION_SERVER_HPP_
```

Action - cpp

action_client.cpp

```
$ cd ~/ros2_ws/src/ros2_cmake/src/
$ code action_client.cpp
```

Action - cpp

action_client.cpp

```cpp
#include "ros2_cmake/action_client.hpp"

FibonacciActionClient::FibonacciActionClient(const rclcpp::NodeOptions & options)
: Node("fibonacci_action_client", options)
{
 this->client_ptr_ = rclcpp_action::create_client<Fibonacci>(
   this,
   "fibonacci");

 this->timer_ = this->create_wall_timer(
   std::chrono::milliseconds(500),
   std::bind(&FibonacciActionClient::send_goal, this));
}
```

Action - cpp

action_client.cpp

```
void FibonacciActionClient::send_goal()
{
 using namespace std::placeholders;
 this->timer_->cancel();
 if (!this->client_ptr_->wait_for_action_server()) {
   RCLCPP_ERROR(this->get_logger(), "Action server not available after waiting");
   rclcpp::shutdown();
 }
 auto goal_msg = Fibonacci::Goal();
 goal_msg.order = 20;
 RCLCPP_INFO(this->get_logger(), "Sending goal");
 auto send_goal_options = rclcpp_action::Client<Fibonacci>::SendGoalOptions();
 send_goal_options.goal_response_callback =
   std::bind(&FibonacciActionClient::goal_response_callback, this, _1);
```

Goal 설정

Action - cpp

action_client.cpp

```
send_goal_options.feedback_callback =
  std::bind(&FibonacciActionClient::feedback_callback, this, _1, _2);
send_goal_options.result_callback =
  std::bind(&FibonacciActionClient::result_callback, this, _1);
this->client_ptr_->async_send_goal(goal_msg, send_goal_options);
}
void FibonacciActionClient::goal_response_callback(const GoalHandleFibonacci::SharedPtr & goal_handle)
{
if (!goal_handle) {
  RCLCPP_ERROR(this->get_logger(), "Goal was rejected by server");
} else {
  RCLCPP_INFO(this->get_logger(), "Goal accepted by server, waiting for result");
}
}
```

Reponse를 받았을 때 실행되는 함수

Reponse 출력

Action - cpp

action_client.cpp

```
void FibonacciActionClient::feedback_callback(
 GoalHandleFibonacci::SharedPtr,
 const std::shared_ptr<const Fibonacci::Feedback> feedback)
{
 std::stringstream ss;
 ss << "Next number in sequence received: ";
 for (auto number : feedback->partial_sequence) {
   ss << number << " ";
 }
 RCLCPP_INFO(this->get_logger(), ss.str().c_str());
}
```

Feedback을 받았을 때 실행되는 함수

Feedback 출력하기

Action - cpp

action_client.cpp

```
void FibonacciActionClient::result_callback(const GoalHandleFibonacci::WrappedResult & result)
{
 switch (result.code) {
   case rclcpp_action::ResultCode::SUCCEEDED:
     break;
   case rclcpp_action::ResultCode::ABORTED:
     RCLCPP_ERROR(this->get_logger(), "Goal was aborted");
     return;
   case rclcpp_action::ResultCode::CANCELED:
     RCLCPP_ERROR(this->get_logger(), "Goal was canceled");
     return;
   default:
     RCLCPP_ERROR(this->get_logger(), "Unknown result code");
     return;
 }
```

Result 받았을 때 실행되는 함수

Result에 따라 결과 처리

Action - cpp

action_client.cpp

```
    std::stringstream ss;
    ss << "Result received: ";
    for (auto number : result.result->sequence) {
      ss << number << " ";
    }
    RCLCPP_INFO(this->get_logger(), ss.str().c_str());
    rclcpp::shutdown();
}
int main(int argc, char * argv[]) {
    rclcpp::init(argc, argv);
    auto action_client = std::make_shared<FibonacciActionClient>();
    rclcpp::spin(action_client);
    rclcpp::shutdown();
    return 0;
}
```

Result 출력

Client 종료

Node 생성

Action - cpp

CMakeList.txt

```
find_package(rclcpp_action REQUIRED)
# action - server
add_executable(action_server src/action_server.cpp)
ament_target_dependencies(action_server rclcpp ros2_interface rclcpp_action)
# action - client
add_executable(action_client src/action_client.cpp)
ament_target_dependencies(action_client rclcpp ros2_interface rclcpp_action)
install(TARGETS
 #action
 action_server
 action_client
 DESTINATION lib/${PROJECT_NAME})
```

Action Server, Client 추가

Action Server, Client 추가

Action - cpp

package.xml

```
<depend>rclcpp_action</depend>
```

Action에 대한 의존성

Action - cpp

Build

```
$ cd ~/ros2_ws/
$ colcon build --packages-select ros2_cmake
$ source install/setup.bash
```

Action - cpp

실행

터미널 1	터미널 2
$ ros2 run ros2_cmake action_server	$ ros2 run ros2_cmake action_client

Action - cpp

실행

터미널 1 출력

```
[INFO] [1735280198.211680051]
[fibonacci_action_server]: Received
goal request with order 20
[INFO] [1735280198.212393409]
[fibonacci_action_server]: Executing
goal
[INFO] [1735280198.212623885]
[fibonacci_action_server]: Publish
feedback
[INFO] [1735280199.212816829]
[fibonacci_action_server]: Publish
feedback
. . .
```

터미널 2 출력

```
[INFO] [1735280198.210899087]
[fibonacci_action_client]: Sending goal
[INFO] [1735280198.212372387]
[fibonacci_action_client]: Goal
accepted by server, waiting for result
[INFO] [1735280198.212821326]
[fibonacci_action_client]: Next number
in sequence received: 0 1 1
[INFO] [1735280199.213190829]
[fibonacci_action_client]: Next number
in sequence received: 0 1 1 2
```

. . . Feedback 출력

Action - cpp

실행

터미널 1 출력

```
[INFO] [1735280372.163757456]
[fibonacci_action_server]: Publish
feedback
[INFO] [1735280373.163730980]
[fibonacci_action_server]: Goal
succeeded
```

터미널 2 출력

```
[INFO] [1735280372.164074256]
[fibonacci_action_client]: Next number
in sequence received: 0 1 1 2 3 5 8 13
21 34 55 89 144 233 377 610 987
1597 2584 4181 6765
[INFO] [1735280373.163960199]
[fibonacci_action_client]: Result
received: 0 1 1 2 3 5 8 13 21 34 55 89
144 233 377 610 987 1597 2584
4181 6765
```

Result 출력



정리하기

ROS2 Topic

Topic 기반 ROS 시스템 구성

- ROS에서 사용하는 Workspace에 대한 개념을 이해하고 파일 시스템을 구성함
- Topic은 기본적인 노드 간 통신 방법으로 노드에서 노드로 Message를 전달함



정리하기

ROS2 Service

Service 기반 ROS 시스템 구성

- Service: 요청이 있을 때 프로세스를 수행하는 방식
- C++와 Python을 이용해 Custom Service를 만들 수 있음



정리하기

ROS2 Action

Action 기반 ROS 시스템 구성

- Action은 Goal(목표로 하는 행위), Feedback(행위가 이루어지는 동안 피드백), Result(결과)로 구성되어 있음
- C++와 Python을 이용해 Custom Action을 만듦